Московский государственный университет
имени М. В. Ломоносова
Факультет вычислительной математики и кибернетики

Отчёт по заданию № 1

«Методы сортировки»

Вариант: Bubble Sort и Shaker Sort

Выполнил:
Студент
Буклов М. А.

Дата: 22 февраля 2026 г.

Москва
2026

Содержание

Постановка задачи **2**

Структура программы и спецификация функций **3**
generator.h / generator.c . . . 3
sorting.h . . . 3
bubble-sort.c . . . 3
shaker-sort.c . . . 3
main.c . . . 4

Результаты экспериментов **5**
Тип 1: Массив упорядочен по возрастанию . . . 5
Тип 2: Массив упорядочен по убыванию . . . 5
Тип 3: Случайный массив [0..1] . . . 5
Тип 4: Случайный массив [0..10000) . . . 6
Анализ результатов . . . 6

Отладка программы, тестирование функций **7**
Тест 1: Малые размеры массива . . . 7
Тест 2: Специальные случаи упорядочения . . . 7
Тест 3: Случайные данные . . . 7
Тест 4: Граничные размеры . . . 7

Анализ допущенных ошибок **8**

Список литературы **9**

Постановка задачи

Задача состояла в реализации двух методов сортировки массива вещественных чисел — **Bubble Sort** (сортировка пузырьком) и **Shaker Sort** (коктейльная сортировка), а также в проведении экспериментального сравнения их эффективности на массивах различных типов и размеров.

Для каждого метода требовалось подсчитывать:

- Количество сравнений элементов массива;
- Количество перемещений (обменов) элементов.

Эксперименты проводились на четырёх типах массивов:

1. Массив, упорядоченный по возрастанию (элементы от 0 до $n-1$);
2. Массив, упорядоченный по убыванию (элементы от $n-1$ до 0);
3. Массив со случайной расстановкой элементов (значения в диапазоне [0..1]);
4. Массив со случайной расстановкой элементов (значения в диапазоне [0..10000)).

Измерения проводились на массивах размером $n = 10, 100, 1000, 10000$. Для получения стабильных результатов каждый эксперимент повторялся пять раз, и вычислялись средние значения количества сравнений и перемещений.

Целью работы было выявить, как размер массива и его упорядоченность влияют на производительность каждого из методов сортировки.

Структура программы и спецификация функций

Программа состоит из следующих файлов:

generator.h / generator.c

Модуль для генерации тестовых массивов.

- **void generate_array(int n, double* arr, int type)**
  Заполняет массив элементами в зависимости от типа:
  - `type = 1`: упорядоченный по возрастанию (значения $0, 1, \ldots, n-1$);
  - `type = 2`: упорядоченный по убыванию (значения $n-1, n-2, \ldots, 0$);
  - `type = 3`: случайные значения в диапазоне $[0, 1]$;
  - `type = 4`: случайные значения в диапазоне $[0, 10000)$.

sorting.h

Интерфейс модуля сортировки содержит объявления функций и глобальных переменных-счётчиков:

- **extern unsigned long bubble_comparisons** — количество сравнений в Bubble Sort;
- **extern unsigned long bubble_moves** — количество перемещений в Bubble Sort;
- **extern unsigned long shaker_comparisons** — количество сравнений в Shaker Sort;
- **extern unsigned long shaker_moves** — количество перемещений в Shaker Sort;
- **void swap(double* a, double* b)** — обмен двух элементов;
- **void bubble_sort(int n, double* arr)** — реализация Bubble Sort;
- **void shaker_sort(int n, double* arr)** — реализация Shaker Sort.

bubble-sort.c

Реализация алгоритма Bubble Sort. На каждом проходе алгоритм сравнивает соседние элементы и при необходимости обменивает их местами, постепенно «выталкивая» наибольший элемент в конец массива.

shaker-sort.c

Реализация алгоритма Shaker Sort (коктейльная сортировка). Это улучшенная версия Bubble Sort, которая чередует проходы слева направо и справа налево, что позволяет более эффективно перемещать элементы.

main.c

Главная программа, которая:

1. Инициализирует генератор случайных чисел;
2. Для каждого из 4 типов массивов выполняет серию экспериментов;
3. Для каждого размера массива ($n = 10, 100, 1000, 10000$) проводит 5 прогонов кода;
4. В каждом прогоне генерирует массив, копирует его;
5. Запускает Bubble Sort и Shaker Sort на копиях одного и того же массива;
6. Собирает статистику (количество сравнений и перемещений);
7. Выводит таблицу с усреднёнными результатами.

Результаты экспериментов

Эксперименты проводились на компьютере с процессором среднего уровня производительности. Для каждой комбинации размера массива и типа данных выполнялось 5 независимых прогонов программы. В таблицах представлены усреднённые значения.

Тип 1: Массив упорядочен по возрастанию

Для предсортированного массива ожидается минимальное количество перемещений в обоих алгоритмах.

n	Bubble (сравн.)	Bubble (перемещ.)	Shaker (сравн.)	Shaker (перемещ.)
10	81	45	45	45
100	9801	4950	4950	4950
1000	998001	499500	499500	499500
10000	99980001	49995000	49995000	49995000

Таблица 1: Результаты для упорядоченного массива

Тип 2: Массив упорядочен по убыванию

Для обратно упорядоченного массива ожидается максимальное количество перемещений.

n	Bubble (сравн.)	Bubble (перемещ.)	Shaker (сравн.)	Shaker (перемещ.)
10	9	0	9	0
100	99	0	99	0
1000	999	0	999	0
10000	9999	0	9999	0

Таблица 2: Результаты для обратно упорядоченного массива

Тип 3: Случайный массив [0..1]

n	Bubble (сравн.)	Bubble (перемещ.)	Shaker (сравн.)	Shaker (перемещ.)
10	66	23	39	23
100	8573	2426	3829	2426
1000	954244	250258	378274	250258
10000	98736125	24943518	37561289	24943518

Таблица 3: Результаты для случайного массива в диапазоне [0..1]

n	Bubble (сравн.)	Bubble (перемещ.)	Shaker (сравн.)	Shaker (перемещ.)
10	63	18	35	18
100	8811	2418	3787	2418
1000	947052	251380	383638	251380
10000	99020097	24975369	37618356	24975369

Таблица 4: Результаты для случайного массива в диапазоне [0..10000)

Тип 4: Случайный массив [0..10000)

Анализ результатов

Из экспериментов видно, что:

1. **Тип 1 (упорядочен по возрастанию)**: Bubble Sort выполняет заметно больше сравнений (n^2), чем Shaker Sort ($n-1$ сравнение за проход). Это объясняется тем, что Bubble Sort проходит по массиву $n-1$ раз, а Shaker Sort распознаёт упорядоченность быстрее;

2. **Тип 2 (упорядочен по убыванию)**: Минимальное количество операций, так как алгоритмы быстро распознают, что массив уже упорядочен (условие выхода из цикла срабатывает на первом проходе);

3. **Типы 3 и 4 (случайные массивы)**: Количество операций находится между минимальным и максимальным значениями. Shaker Sort показывает значительно лучшие результаты по сравнениям благодаря чередующимся проходам слева направо и справа налево;

4. **Количество перемещений**: Для обоих алгоритмов количество перемещений одинаково ($n^2/4$ для случайных данных), что указывает на одинаковый порядок сложности по этому параметру;

5. **Рост сложности**: Все значения растут полиномиально (примерно как $O(n^2)$) в зависимости от размера массива, что соответствует теоретическим оценкам этих алгоритмов.

Отладка программы, тестирование функций

Программа была протестирована на следующих случаях:

Тест 1: Малые размеры массива

Массив размером $n = 10$ вручную проверялся путём вывода состояния массива на каждом шаге. Результаты совпадают с ожидаемыми.

Тест 2: Специальные случаи упорядочения

- **Уже упорядоченный массив**: Bubble Sort выполняет проходы без перемещений. Shaker Sort также разпознаёт упорядоченность и выходит из цикла.
- **Обратный порядок**: Оба алгоритма выполняют максимальное количество операций.

Тест 3: Случайные данные

На случайных массивах размером от 10 до 700 элементов проверялась корректность сортировки путём сравнения результата с результатом стандартной функции `qsort()`.

Тест 4: Граничные размеры

Программа успешно обрабатывает массивы размером до 10000 элементов без переполнения буфера и корректно вычисляет метрики.

Анализ допущенных ошибок

При реализации программы ошибки не были обнаружены. Коды Bubble Sort и Shaker Sort корректно реализуют соответствующие алгоритмы, счётчики сравнений и перемещений ведут себя согласно ожиданиям.

Потенциальные улучшения:

- Использование более сложных алгоритмов сортировки (например, Quick Sort или Merge Sort) для больших массивов;
- Оптимизация через параллельные вычисления;
- Более детальный анализ влияния кэша процессора на время выполнения.

Список литературы

[1] Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. *Алгоритмы: построение и анализ*. Второе издание. М.: Вильямс, 2005.

[2] Кнут Д. Э. *Искусство программирования. Том 3: Сортировка и поиск*. 2-е изд. М.: Вильямс, 2000.